

03

SOFTWAREENTWICKLUNG

KURSinHALTE UND TERMINE

Kursinhalte

Data Science	1
Statistische Grundlagen	2
Softwareentwicklung	3
Testing / Integration / Deployment	4
Ansätze, Methoden und Anwendungen Künstlicher Intelligenz (KI)	5-8
Zusammenfassung / Fragen / Klausurvorbereitung	9

Termine

#	Wochentag	Datum	von - bis	Räume
1	Freitag	04.04.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
2	Freitag	11.04.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.27 Bothfeld
3	Freitag	25.04.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
4	Freitag	16.05.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
5	Freitag	23.05.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
6	Freitag	06.06.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
7	Freitag	20.06.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
8	Freitag	04.07.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
9	Freitag	11.07.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt

Softwareentwicklung

Lernziele



Nach der Bearbeitung dieser Lektion werdet ihr wissen, ...

- wie sich **klassische** und **agile Projektmanagement-Ansätze** in der **Softwareentwicklung** unterscheiden.
- wie sich **agile Projektmanagement-Ansätze** in der **Softwareentwicklung anwenden** lassen (bspw. **Kanban, Scrum, Scrumban**).
- was **Extreme Programming (XP)** bedeutet und welche **Kernpraktiken** dazugehören.
- wie sich **Extreme Programming (XP)** in Data Science Projekten **anwenden** lässt.

SOFTWAREENTWICKLUNG DEFINITIONEN

Software-Produkt

„...ist die Gesamtheit von Softwarekomponenten (Programme, Dokumentationen, usw.), die als Ganzes entwickelt, vertrieben, angewendet und gewartet werden.“
(Dumke, 2003, S. 4)

Das Software-Produkt soll in systematischer Form mittels eines Prozesses realisiert werden. Software-Produkte werden dabei üblicherweise durch Software-Projekte realisiert.

Software-Projekt

„...ist der konkrete Prozess zur Entwicklung eines konkreten Software-Produktes mit den dafür notwendigen Ressourcen.“ (Dumke, 2003, S. 9)

- Einmaliges Vorhaben, zeitlich begrenzt, üblicherweise mit definiertem Endtermin
- Klar definierte Ziele und Zusammenarbeitsformen
- Spezielles Budget
- Projektverlauf birgt Risiken



SOFTWAREENTWICKLUNG DEFINITIONEN

Entwicklungs-Ressourcen

„...alle für die Software-Entwicklung aufgewendeten bzw. aufzuwendenden personellen, Software- und Hardware-Mittel...“ (Dumke, 2003, S. 8)

Personelle Ressourcen können nach den folgenden Kriterien unterschieden werden:

- **Aufgabenverteilung**
(bspw. Entwickler, Applicationmanagement, Support-Mitarbeiter, Qualitätsmanager, Projektmanager)
- **Phase in der Softwareentwicklung**
(bspw. Requirements Engineer, Designer, Programmierer, Tester, Betriebsmitarbeiter)
- **Spezialkenntnisse**
(bspw. Multimedia-Designer, Netz-Administrator, Data Scientist)
- **Erfahrung**
(junior, senior, Guru 😊)



SOFTWAREENTWICKLUNG DEFINITIONEN

Software-Entwicklungsprozess

„... ist der gesamte Prozess der Aufgabenstellung, Planung, Realisierung und Bewertung einer Software-/Hardware-Anwendung einschließlich der verwendeten Hilfsmittel und Methoden und dem erforderlichen Personal.“ (Dumke, 2003, S. 6)

Impliziert folgende Merkmale:

- Aufgabenstellung erfordert Analyse
- Es sind mehrere Personen beteiligt
- Wird für einen anderen Nutzer entwickelt
- Ergebnis enthält computergestützte Komponenten als auch organisatorische Richtlinien zur Einführung und Anwendung



Jedes Software Produkt durchlebt verschiedene Stadien und benötigt die dabei erforderlichen Ressourcen

Software-Lebenszyklus

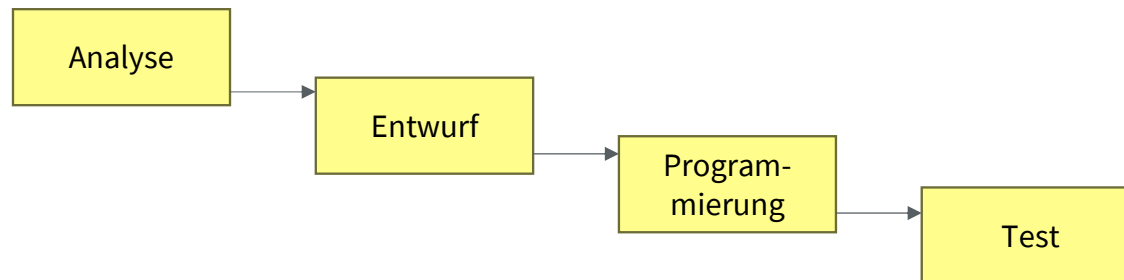
„...ist der Prozess der Entwicklung von Software-Produkten und kennzeichnet alle Phasen und Stadien dieser Produkte von ihrer Entwicklung, Einführung und Wartung bis zu ihrer Ablösung und Beseitigung“ (Dumke, 2003, S. 18)

SOFTWAREENTWICKLUNG

KLASSISCHE PROJEKTMANAGEMENT-ANSÄTZE

Beim klassischen Projektmanagement in der Softwareentwicklung werden die verschiedenen Entwicklungsaktivitäten mittels Phasen nacheinander durchlaufen und von entsprechenden Experten für die jeweilige Phase umgesetzt. Zwischen den Phasen finden Übergaben statt und es wird angestrebt Rücksprünge zu vorherigen Phasen möglichst zu vermeiden.

Ein extremes Beispiel mit streng sequentiellen Phasen ist das „**Wasserfall-Modell**“



Quelle: Vgl. Laudon et. al, 2016, S 890.

Die Phasen können weiter untergliedert werden (bspw. Entwurf in Software- und Systementwurfsphase).

SOFTWAREENTWICKLUNG

KLASSISCHE PROJEKTMANAGEMENT-ANSÄTZE

Wasserfall-Modell und kritische Betrachtung

- Älteres Modell, wird jedoch nach wie vor für mittlere bis große Entwicklungsprojekte eingesetzt
- Idee basiert auf einer streng sequentiellen Ablauforganisation ohne Rücksprünge
- In der Praxis ist eine solche Ablauforganisation jedoch kaum durchzuhalten, daraus sind Varianten entstanden, die Rücksprünge zulassen
- Kontroll- und Entscheidungspunkte am Ende einer Aktivität (führt zu viel Dokumentation)
- Beginn der nächsten Aktivität nur bei positiver Überprüfung („Stage-Gate-Modell“)
- Die klassische Softwareentwicklung folgt einem „Stage-Gate-Modell“, dabei wird der Übergang in die nächste Phase durch einen Gatekeeper ausführlich geprüft, um Rücksprünge zu vermeiden.
- Unterschiedliche Experten für die verschiedenen Phasen erfordern aufwändige Übergaben
- Die Dokumentation wird dabei zu einem wichtigen Kommunikationsmittel.
- Problem wenn Anforderungen in der Analysephase übersehen oder falsch verstanden wurden
- Sofern Änderungsbedarf in späteren Phasen auftritt, sind diese häufig mit hohem Aufwand verbunden, da alle vorgelagerten Phasen erneut durchlaufen werden müssen.
- Dem Anforderungsmanagement kommt demnach eine besondere Rolle zu.



Quelle: Wirtschaftsinformatik, Laudon et. al, 2016, S.889ff.

SOFTWAREENTWICKLUNG

KLASSISCHE PROJEKTMANAGEMENT-ANSÄTZE

Anforderungsmanagement

- Die Benutzer sind nur eingeschränkt am Prozess beteiligt
- Die Bedürfnisse der Benutzer müssen in Form von Anforderungen dokumentiert und im Prozess weitergegeben werden
- In der Analyse werden sie vom Systemanalytiker zu den Anforderungen befragt und im Akzeptanztest überprüfen sie das Ergebnis
- Häufig wird durch den Auftraggeber eine detaillierte Leistungsbeschreibung erstellt („Lastenheft“)
- Entwurfsmitarbeiter erfahren durch die Anforderungsspezifikation, was das neue System leisten soll
- Sofern es sich um eine Ausschreibung handelt, wird von den potentiellen Auftragnehmern eine Beschreibung der Leistungen („Pflichtenheft“) mit einem zugehörigen Preis erstellt
- Programmierer erfahren durch die Entwurfsdokumente, welche Funktionen und Module sie programmieren müssen



Quelle: Wirtschaftsinformatik, Laudon et. al, 2016, S.889ff.

SOFTWAREENTWICKLUNG PROJEKTMANAGEMENT-ANSÄTZE

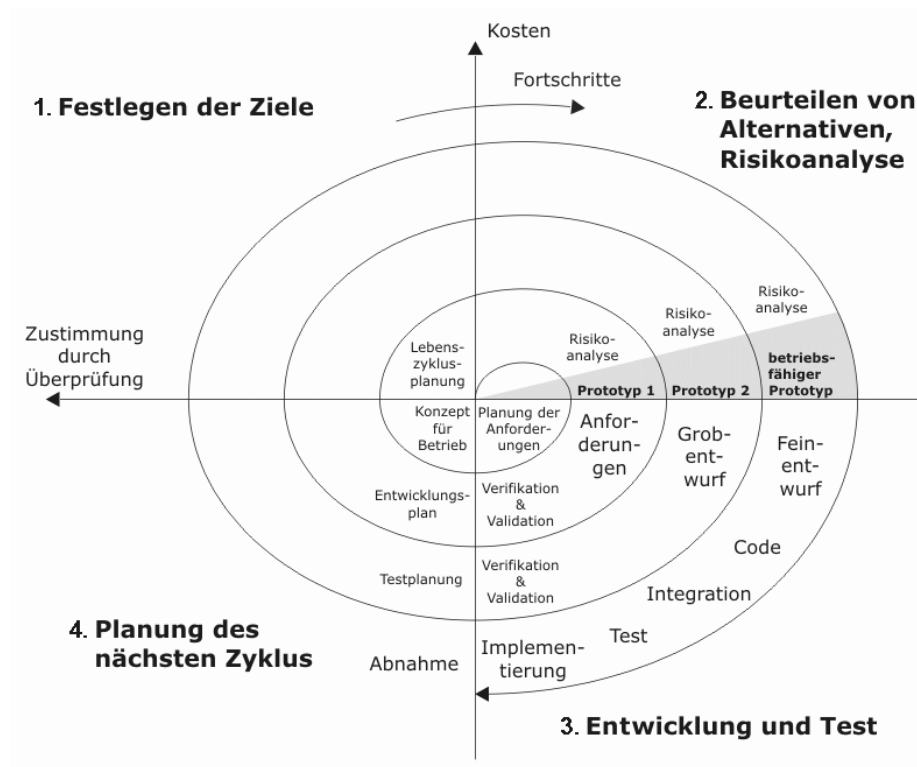
Varianten und weitere Modelle

- Wasserfall-Modelle mit mehreren Phasen
- Spiralmodell
- V-Modell
- Prototyping
- Inkrementelle Entwicklung
- Rapid Application Development (RAD)
- Rational Unified Process (RUP)
- Agile Softwareentwicklung (Kanban, SCRUM, SAFe)
- Extreme Programming (XP)
- ...

SOFTWAREENTWICKLUNG SPIRALMODELL

Spiralmodell von Boehm

„...strukturiert den Entwicklungsprozess ebenfalls in eine Folge von Phasen. Zu Beginn jeder Phase werden in Abhängigkeit von den verfügbaren Mitteln und den identifizierten Risiken das Entwicklungsziel und die Maßnahmen der Phase festgelegt. Die Phase endet mit der Planung des weiteren Vorgehens in den Folgephasen.“
(Laudon et al., 2016, S. 895.)

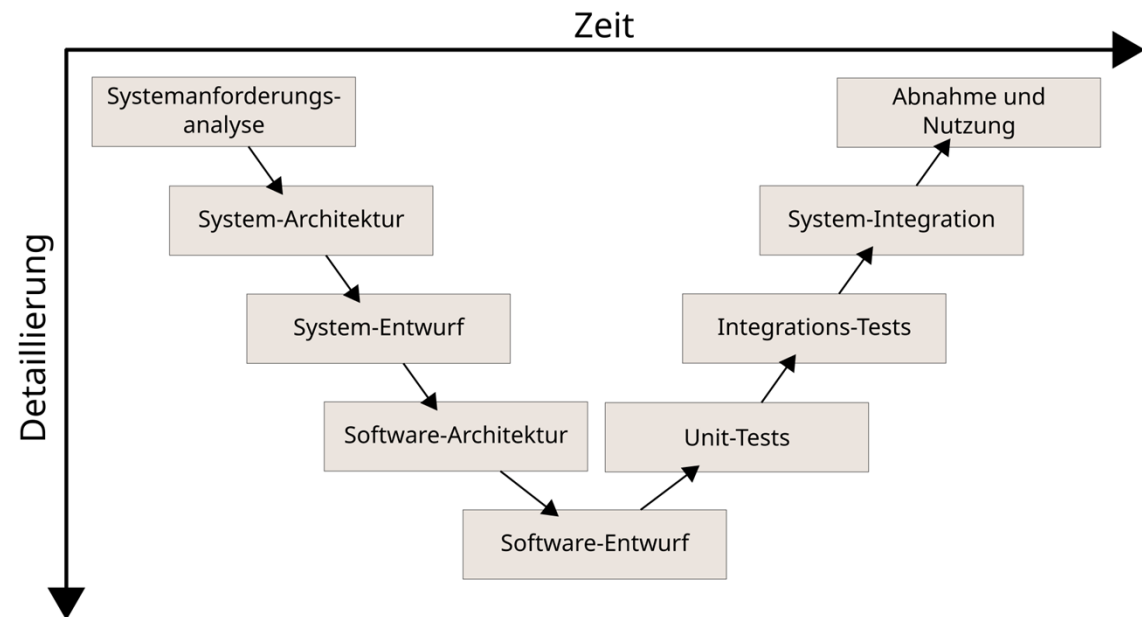


Quelle: Spiralmodell nach Boehm

SOFTWAREENTWICKLUNG

V-MODELL

„Das **V-Modell** beschreibt ein sequentielles Vorgehensmodell der Software- und Systementwicklung, das Wert auf die frühe Planung und spätere Verifikation und Validierung legt. Die linke Seite des V steht für die Spezifikation, die rechte für die Teststufen.“ (Balzert, 2001)



Quelle: <https://de.wikipedia.org/wiki/V-Modell>

SOFTWAREENTWICKLUNG

PROTOTYPING

Prototyping

„Der Prozess, schnell und kostengünstig ein experimentelles System aufzubauen, das den Benutzern als Demonstration und für Auswertungszwecke bereitgestellt wird, sodass diese ihre Anforderungen besser einschätzen können.“ (Laudon et al., 2016, S. 893.)

Schritte:

1. Identifizierung der grundlegenden Anforderungen des Benutzers
2. Entwicklung eines anfänglichen Prototyps
3. Einsatz des Prototyps
4. Überarbeitung und Erweiterung des Prototyps

Wegwerf-Prototyp

„Die vorläufige Arbeitsversion eines Informationssystems, das zu Demonstrations- und Auswertungszwecken dient.“ (Laudon et al., 2016, S. 893.)

Quelle: Wirtschaftsinformatik, Laudon et. al, 2016, S.893..

SOFTWAREENTWICKLUNG

KLASSISCHE PROJEKTMANAGEMENT-ANSÄTZE

Rational Unified Process (RUP)

- Basiert auf dem „Unified Process“, einem Metamodell für Vorgehensmodelle zur Softwareentwicklung
- RUP ist eine Implementierung des Unified Process und basiert auf den Prinzipien:
 - Orientierung an Use Cases
 - Fokus auf Architektur und Risiken
 - Inkrementelles und iteratives Vorgehen
- Von der Firma Rational Software (seit 2003 Teil von IBM) entwickelt
- Neben dem Modell gibt es auch eine zugehörige Software von IBM

Phasen:

1. Inception (Konzeption) – Ziel: Projektumfang und grobe Anforderungen klären
2. Elaboration (Ausarbeitung) – Ziel: Architektur und kritische Risiken klären
3. Construction (Konstruktion) – Ziel: System implementieren
4. Transition (Übergang) – Ziel: System ausrollen, Nutzer schulen

Quelle: <https://www.redbooks.ibm.com/abstracts/SG247244.html?Open>

SOFTWAREENTWICKLUNG

KLASSISCHE PROJEKTMANAGEMENT-ANSÄTZE

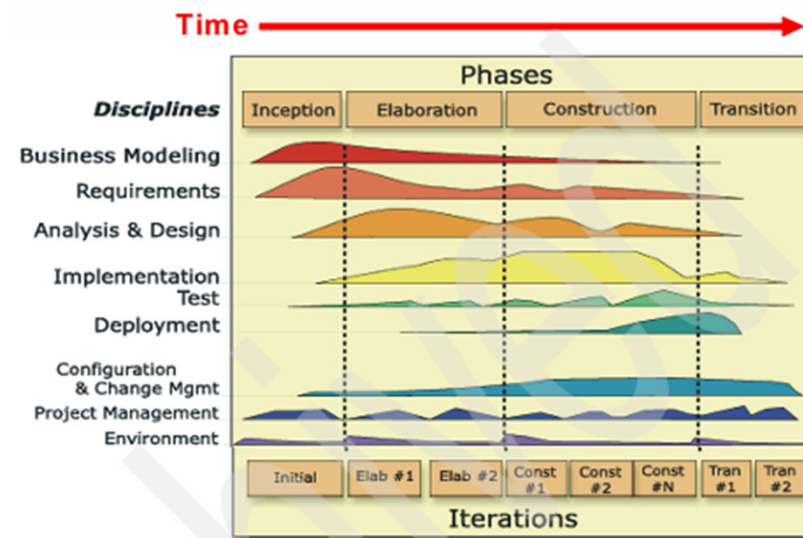
Rational Unified Process (RUP)

Phasen:

1. Inception (Konzeption) – Ziel: Projektumfang und grobe Anforderungen klären
2. Elaboration (Ausarbeitung) – Ziel: Architektur und kritische Risiken klären
3. Construction (Konstruktion) – Ziel: System implementieren
4. Transition (Übergang) – Ziel: System ausrollen, Nutzer schulen

Disziplinen:

1. Unternehmensplan/Geschäftsmodell
2. Anforderungen
3. Analyse & Entwurf
4. Implementierung
5. Test
6. Bereitstellung
7. Konfiguration / Änderungsmanagement
8. Projektmanagement
9. Umgebungen



Quelle: IBM Redbook, 2006, S 21.

Softwareentwicklung klassische Projektmanagement-Ansätze

Übungsfragen



1. Wie **unterscheidet sich** ein **Software-Produkt** von einem **Software-Projekt**?
2. Nach welchen **Kriterien** lassen sich **personelle Ressourcen** unterscheiden?
3. Was ist das **Wasserfall-Modell**?
4. Welche **Kritikpunkte** gibt es am **Wasserfall-Modell**?
5. Was wird unter einem „**Lastenheft**“ und unter einem „**Pflichtenheft**“ verstanden?
6. **Warum** ist es wichtig ein gutes **Anforderungsmanagement** zu haben?
7. Welche **klassischen Ansätze** der **Softwareentwicklung** gibt es darüber hinaus noch? **Nenne 3 Ansätze.**

SOFTWAREENTWICKLUNG

AGILE PROJEKTMANAGEMENT-ANSÄTZE

„**Agile Softwareentwicklung** ist ein Sammelbegriff für eine Reihe von Methoden und Praktiken, die auf Werten und Prinzipien des Manifests Agiler Softwareentwicklung basieren.“

– **Agile Alliance, 2018**

Agile Leitsätze

Vier Leitsätze wurden im Februar 2001 als Agiles Manifest (englisch Manifesto for Agile Software Development oder kurz Agile Manifesto) formuliert:

„Wir erschließen bessere Wege, Software zu entwickeln, indem wir es selbst tun und anderen dabei helfen. Durch diese Tätigkeit haben wir diese Werte zu schätzen gelernt:

1. **Individuen und Interaktionen** sind wichtiger als **Prozesse und Werkzeuge**
2. **Funktionierende Software** ist wichtiger als **umfassende Dokumentationen**
3. **Zusammenarbeit mit dem Kunden** ist wichtiger als **Vertragsverhandlungen**
4. **Reagieren auf Veränderung** ist wichtiger als das **Befolgen eines Plans**

Das heißt, obwohl wir die Werte auf der rechten Seite wichtig finden, schätzen wir die Werte auf der linken Seite höher ein.“

Quelle: <https://scrumguides.org/scrum-guide.html#purpose-of-the-scrum-guide>

SOFTWAREENTWICKLUNG

AGILE PROJEKTMANAGEMENT-ANSÄTZE

SCRUM

“Scrum is a lightweight framework that helps people, teams and organizations generate value through adaptive solutions for complex problems.”

In a nutshell, Scrum requires a Scrum Master to foster an environment where:

1. A **Product Owner** orders the work for a complex problem into a **Product Backlog**.
2. The **Scrum Team** turns a selection of the work into an **Increment of value** during a Sprint.
3. The **Scrum Team** and its **stakeholders inspect** the results and **adjust** for the next Sprint.
4. *Repeat*

(<https://scrumguides.org/scrum-guide.html#purpose-of-the-scrum-guide>)



Ziel:

Ein **funktionsfähiges Produktinkrement** am Ende jedes Sprints, **schnelles Kundenfeedback** und **kontinuierliche Verbesserung** des Teams und Produkts.

SOFTWAREENTWICKLUNG

AGILE PROJEKTMANAGEMENT-ANSÄTZE

SCRUM - Kernelemente

- **Rollen:**

- **Product Owner** – verantwortet den Produkterfolg und priorisiert Anforderungen (Product Backlog)
- **Scrum Master** – unterstützt das Team, beseitigt Hindernisse und fördert die Einhaltung von Scrum
- **Entwicklungsteam** – interdisziplinär, selbstorganisiert und verantwortlich für die Umsetzung

- **Ereignisse (Events):**

- **Sprint** – ein fester Entwicklungszyklus (meist 2–4 Wochen)
- **Sprint Planning** – Planung der Sprint-Ziele und Aufgaben
- **Daily Scrum** – tägliches 15-Minuten-Meeting zur Abstimmung
- **Sprint Review** – Vorstellung der Ergebnisse mit Feedback
- **Sprint Retrospective** – Rückblick zur Verbesserung der Zusammenarbeit

- **Artefakte:**

- **Product Backlog** – Liste aller gewünschten Funktionen/Anforderungen
- **Sprint Backlog** – Aufgaben, die im aktuellen Sprint umgesetzt werden
- **Increment** – fertiges, potenziell auslieferbares Produktstück



Quelle: <https://scrumguides.org/scrum-guide.html#purpose-of-the-scrum-guide>

SOFTWAREENTWICKLUNG

AGILE PROJEKTMANAGEMENT-ANSÄTZE

KANBAN

Kanban ist eine **agile Methode** zur **Visualisierung und Steuerung von Arbeitsprozessen**. Es hilft Teams, den **Workflow zu optimieren**, **Engpässe zu erkennen** und die **Liefergeschwindigkeit zu verbessern**.

Grundprinzipien:

- 1. Visualisiere die Arbeit** z. B. mit einem Kanban-Board (Spalten wie *To Do*, *In Progress*, *Done*)
- 2. Begrenze parallele Arbeit (WIP-Limits)** um Überlastung zu vermeiden und den Fokus zu erhöhen
- 3. Manage den Fluss** Ziel: gleichmäßiger Arbeitsfluss vom Anfang bis zum Ende
- 4. Make Prozesse und Regeln explizit** Transparenz im Team schaffen
- 5. Verbessere kontinuierlich (Kaizen)** regelmäßige Anpassung und Optimierung des Workflows

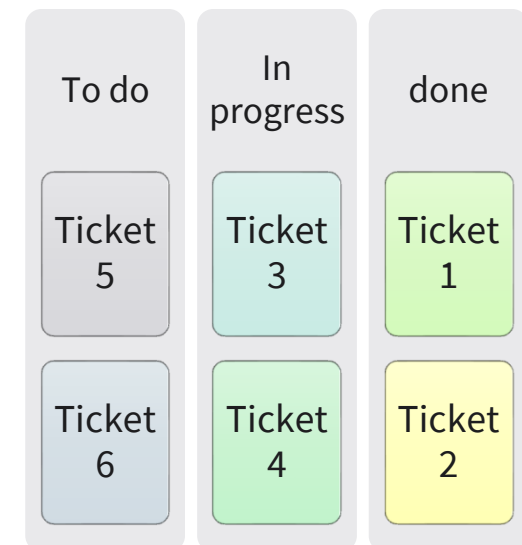
Besonderheiten:

- Kein fester Sprint – **kontinuierliche Lieferung**
- **Ideal für Wartungsteams, Support oder kontinuierliche Projekte**
- Flexibel und **gut kombinierbar mit anderen Methoden**

Ziel:

Effizienter arbeiten, Durchlaufzeiten verkürzen, Engpässe früh erkennen und Prozesse laufend verbessern.

Quelle: Wirtschaftsinformatik, Laudon et. al, 2016.



AGILE PROJEKTMANAGEMENT-ANSÄTZE

KANBAN TAFEL

Was ist eine Kanban-Tafel?

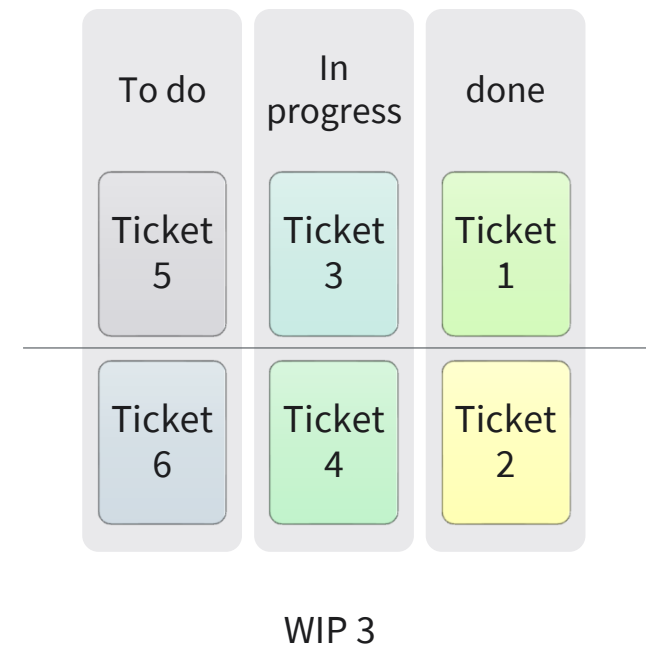
- Eine **Kanban-Tafel** ist ein **visuelles Werkzeug**, das hilft, den Arbeitsprozess eines Teams darzustellen und zu steuern. Sie zeigt, **welche Aufgaben** gerade anstehen, bearbeitet werden oder bereits erledigt sind.

Typischer Aufbau einer Kanban-Tafel:

- Die Tafel ist in **Spalten** unterteilt, die den **Workflow** abbilden. Ein einfaches Beispiel:

Bestandteile:

- **Karten (Tasks):** Jede Aufgabe wird als **Karte** dargestellt (z. B. ein Zettel oder digitales Ticket).
- **Spalten:** Zeigen den **Status** der Aufgabe im Arbeitsprozess.
- **Swimlanes (optional):** Horizontale Unterteilungen, z. B. für verschiedene Projekte, Teammitglieder oder Prioritäten.
- **WIP-Limits (optional):** Begrenzung, wie viele Aufgaben gleichzeitig in einer Spalte sein dürfen (z. B. max. 3 in „In Progress“).



AGILE PROJEKTMANAGEMENT-ANSÄTZE

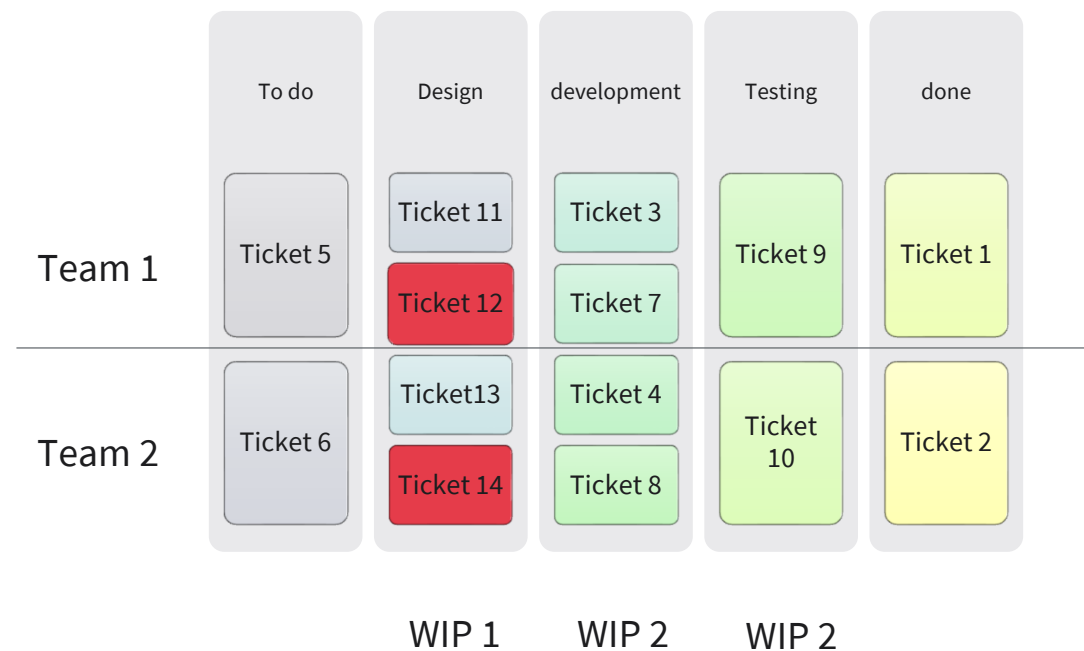
KANBAN TAFEL

Ziele der Kanban-Tafel:

- **Transparenz:** Alle sehen auf einen Blick, woran gearbeitet wird.
- **Fokus:** Begrenzung der parallelen Arbeit (WIP) verhindert Überlastung.
- **Fluss verbessern:** Engpässe im Prozess werden früh erkannt.
- **Kontinuierliche Verbesserung:** Durch Beobachtung und Anpassung.

Tools:

- Digitale Varianten z. B. mit **Trello, Jira, Asana, Azure Boards** oder **Miro**.



AGILE PROJEKTMANAGEMENT-ANSÄTZE

SCRUMBAN

„Die **Scrumban-Methode** kombiniert die besten Funktionen von Scrum und Kanban in einem hybriden Projektmanagement-Framework. Dabei wird die stabile Struktur von Scrum aus Sprints, Stand-ups und Retrospektiven genutzt. Hinzu kommen der visuelle Workflow und die Work-in-Progress-Einschränkungen von Kanban. Das Ergebnis ist eine wirklich flexible Methode zur Verwaltung von Projekten jeder Größe.“

Elemente aus Scrum

- Sprints
- Tägliche Stand-ups
- Retrospektiven

Elemente aus Kanban:

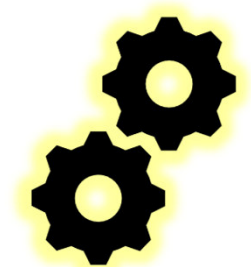
- Kanban Board
- Karten
- Work-in-progress (WIP) Limit

Vorteile Scrumban:

- Höhere Flexibilität (Änderungen im Sprint möglich)
- Continuous Delivery
- Weniger Überlastung (WIP)
- Schnellere Problemlösung (Transparenz mittels Karten)
- Auch für komplexe Projekte geeignet (kontinuierlich und schrittweise Verbesserung)

Nachteile Scrumban:

- Vagheit (relativ neu und weniger Dokumentation dazu)
- Weniger Kontrolle (verzichtet auf Scrum-Rollen, Team verwaltet sich selbst)
- Komplexität (Elemente aus 2 Methoden kann zu Verwirrung führen)



AGILE PROJEKTMANAGEMENT-ANSÄTZE

VERGLEICH DER METHODEN

	Scrum	Kanban	Scrumban
Methode	• Sprints mit fester Länge • Feste Rollen • Konsistente Lieferung	• Work in Progress einschränken • Tasks visuell nachverfolgen • Kontinuierliche Abläufe	• Sprints mit fester Länge • Work in Progress einschränken - Aufgaben visuell nachverfolgen - Kontinuierliche Abläufe
Rollen	• Produktinhaber • Scrum Master • Entwicklerteam	• Keine Angaben	• Keine Angaben
Artefakte	• Produkt-Backlog • Sprint-Backlog • Inkrement abgeschlossen	• Kanban Board - Kanban-Karten	• Scrumban Board • Scrumban-Karten
Veranstaltungen	• Sprintplanung • Tägliches Stand-up • Sprint-Review • Sprint-Retrospektive	• Kanban-Meeting	• Sprintplanung • Tägliches Stand-up • Sprint-Retrospektive
Prozessablauf	• Produkt-Backlog • Sprint-Backlog • In Bearbeitung • Überprüfung - Erledigt	• Zu erledigen • In Bearbeitung • Erledigt	• Zu erledigen • In Bearbeitung • Erledigt

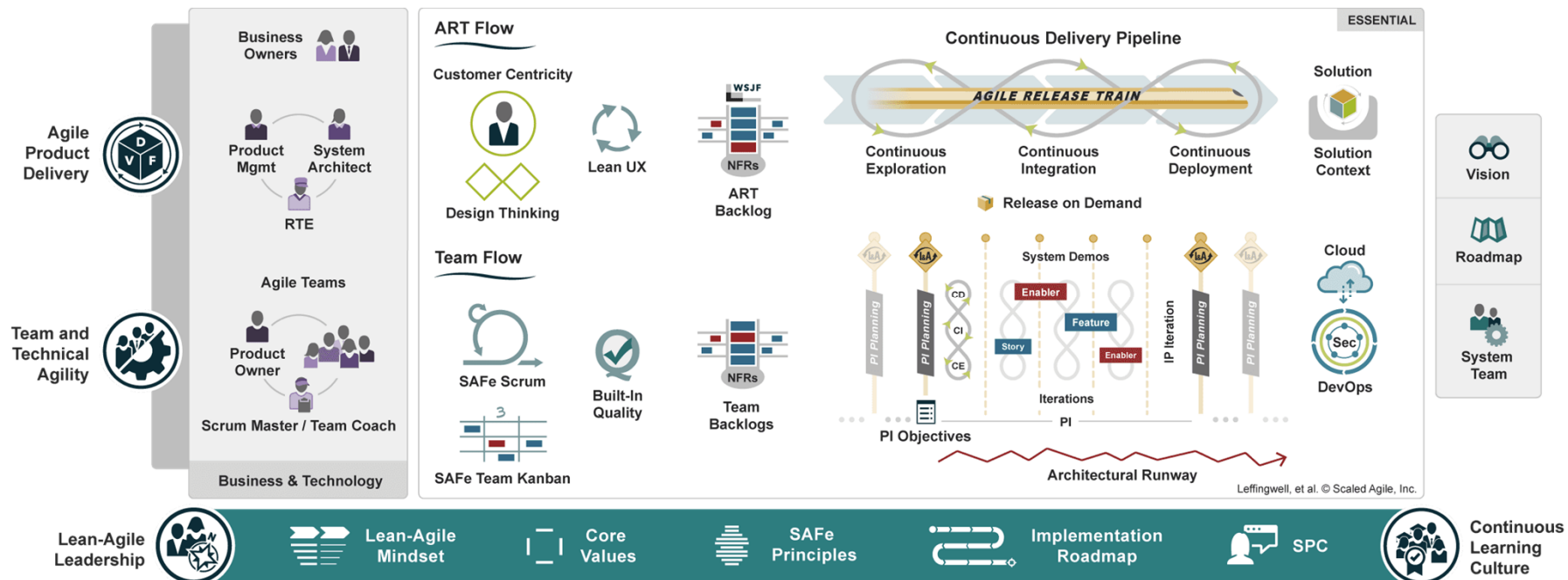
Quelle: <https://www.atlassian.com/de/agile/project-management/scrumban>

AGILE PROJEKTMANAGEMENT-ANSÄTZE

SCALED AGILE FRAMEWORK (SAFE)

Scaled Agile Framework (SAFe)

- Framework für große Projekte, bei denen mehrere Teams gemeinsam an einer Lösung arbeiten
- Framework beinhaltet Rollen, Prozesse und Methoden zur Zusammenarbeit
- <https://framework.scaledagile.com/>



Quelle: <https://framework.scaledagile.com/>

AGILE PROJEKTMANAGEMENT-ANSÄTZE

SCALED AGILE FRAMEWORK (SAFE)

Seven Core Competencies of Business Agility

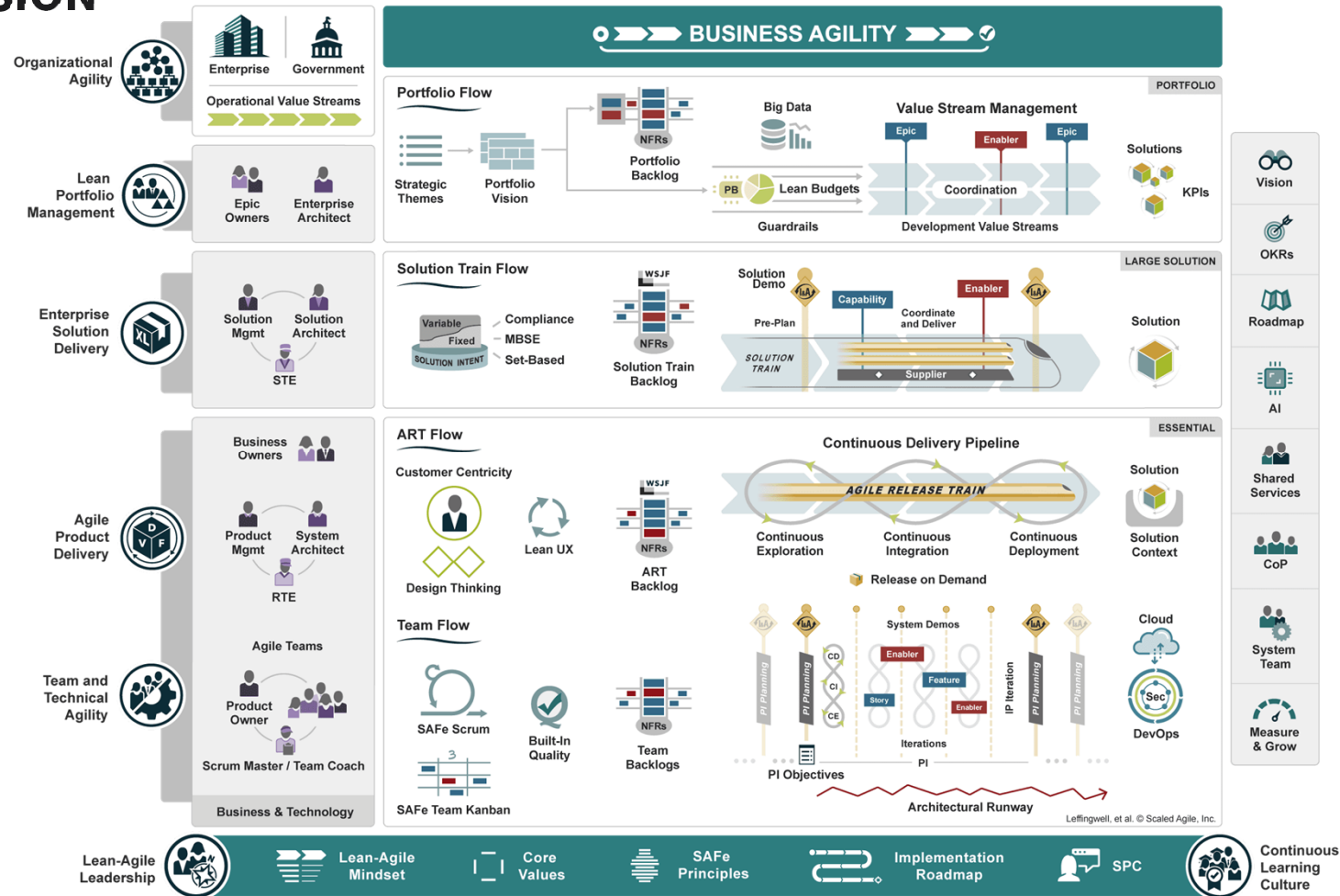
- 1. Lean-Agile Leadership**
Führungskräfte leben Lean-Agile-Prinzipien vor, geben klare Visionen vor, schaffen ein Umfeld für selbstorganisierte Teams und treiben Veränderung aktiv voran.
- 2. Continuous Learning Culture**
Organisationen fördern eine Kultur des lebenslangen Lernens, Experiments und der Innovation
- 3. Team and Technical Agility**
Teams arbeiten nach agilen Prinzipien (Scrum, Kanban etc.) und setzen technische Exzellenz (z. B. Clean Code, CI/CD, Testautomatisierung) um.
- 4. Agile Product Delivery**
Fokus auf **kundenzentrierte Entwicklung** mit schneller, kontinuierlicher Lieferung von wertvollen Produkten
- 5. Enterprise Solution Delivery**
Fähigkeit, große, komplexe Systeme (z. B. Industrie-Software, Infrastrukturen, Plattformen) **agil zu entwickeln, integrieren und auszuliefern**.
- 6. Lean Portfolio Management**
Strategische Ausrichtung und Finanzierung ganzer Wertströme anstatt einzelner Projekte – mit agilen Methoden im Portfolio-Management.
- 7. Organizational Agility**
Organisationen passen sich schnell an Marktveränderungen an, richten Prozesse und Strukturen kontinuierlich neu aus.



Quelle: <https://framework.scaledagile.com/>

AGILE PROJEKTMANAGEMENT-ANSÄTZE

SAFE FULL VERSION



Quelle: <https://framework.scaledagile.com/>

Softwareentwicklung agile Projektmanagement-Ansätze

Übungsfragen



1. Was versteht man unter **agiler Softwareentwicklung**?
2. Nenne die 4 **agilen Leitsätze**!
3. Was ist das **Ziel von Scrum**?
4. Welches sind die wesentlichen **Merkmale von Scrum**?
5. Was ist das **Ziel von Kanban**?
6. Welches sind die wesentlichen **Merkmale von Kanban**?
7. Was sind die **Vorteile von Scrumban**?
8. Was sind die **Nachteile von Scrumban**?
9. Wofür wird das **Scaled Agile Framework (SAFe)** verwendet?

Softwareentwicklung

Lernziele



Nach der Bearbeitung dieser Lektion werdet ihr wissen, ...

- wie sich **klassische** und **agile Projektmanagement-Ansätze** in der **Softwareentwicklung** unterscheiden.
- wie sich **agile Projektmanagement-Ansätze** in der **Softwareentwicklung anwenden** lassen (bspw. **Kanban, Scrum, Scrumban**).
- was **Extreme Programming (XP)** bedeutet und welche **Kernpraktiken** dazugehören.
- wie sich **Extreme Programming (XP)** in Data Science Projekten **anwenden** lässt.

30 Minuten Zeit



Aufgabe:

1. Bildet **Teams aus 2 Personen**.
 - eine Person übernimmt die Doppelrolle „Requirements Engineer“ (**RE**) und „Tester“
 - die andere Person übernimmt die Rolle „Developer“ (**DEV**)
2. Erstellt gemeinsam ein **simples Kanban-Board** (Tool eurer Wahl).
3. Erstellt arbeitsteilig eine einfache **explorative Datenanalyse (EDA)** für Immobilien in Kalifornien (Datensatz „cali_data.csv“).
 - Der **RE** erstellt mindestens 2 Karten für euer Projekt (in Form von „User Stories“)
 - Der **DEV** setzt die Stories um und aktualisiert das Board
 - Der **RE** testet die Stories

Tipp: Teile der **EDA** sind Charakteristika wie **Skalenniveau**, **statistische Verteilung** und **deskriptive Statistiken**, **Visualisierungen** (**Histogramm**, **Boxplots**, **Streudiagramm**), Evaluation der **Datenqualität** auf **Anomalien**, **Ausreißer**, **Duplikate**, **fehlende Werte**, erste Zusammenhänge zwischen den Attributen / Features anhand einer **Heatmap**.



Softwareentwicklung

Lernziele



Nach der Bearbeitung dieser Lektion werdet ihr wissen, ...

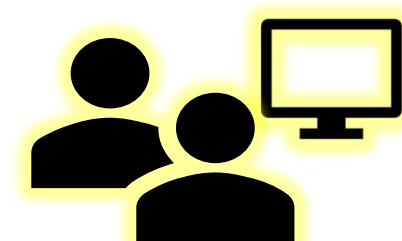
- wie sich **klassische** und **agile Projektmanagement-Ansätze** in der **Softwareentwicklung** unterscheiden.
- wie sich **agile Projektmanagement-Ansätze** in der **Softwareentwicklung anwenden** lassen (bspw. **Kanban, Scrum, Scrumban**).
- was **Extreme Programming (XP)** bedeutet und welche **Kernpraktiken** dazugehören.
- wie sich **Extreme Programming (XP)** in Data Science Projekten **anwenden** lässt.

SOFTWAREENTWICKLUNG

EXTREME PROGRAMMING (XP)

„**Extreme Programming (XP)** versucht, durch kleine Entwicklungsteams und häufige Iterationen, die Aufgaben so klein und überschaubar zu halten, dass Anforderungs- und Entwurfsdokumente überflüssig werden, der Programmcode als Dokumentation ausreicht und so die ausufernden Dokumentationspflichten der Entwickler vermieden werden können.“ (Laudon, 2016, S. 902)

- Betont die Programmiertätigkeit
- Kunde sollte während der Projektlaufzeit für Rückfragen erreichbar ist
- Kleine Teams, häufige Iterationen, Aufgaben überschaubar halten
- Anforderungs- und Entwicklungsdokumentation überflüssig bzw. in Form des Programmiercodes
- Spezielle Arbeitsweisen wie Pair Programming und testgetriebene Entwicklung sollen Qualitätsrisiken reduziert und die Kompetenz der Entwickler erhöht werden
- Immer komplexer werdende Managementprozesse in Entwicklungsprojekten soll dadurch insbesondere in kleinen und mittleren Projekten entgegengewirkt werden
- der Ansatz wird durchaus kontrovers diskutiert, ein genereller Erfolg des Ansatzes ist nicht belegt



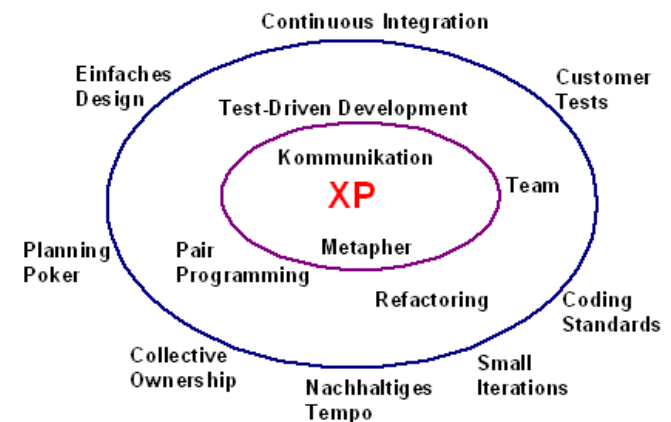
Quelle: Wirtschaftsinformatik, Laudon et. al, 2016.

SOFTWAREENTWICKLUNG

EXTREME PROGRAMMING (XP)

Kernpraktiken von Extreme Programming (XP)

- **Test Driven Development (TDD)**
Tests werden vor dem Code geschrieben.
- **Pair Programming**
Zwei Entwickler arbeiten gemeinsam an einem Arbeitsplatz.
- **Continuous Integration**
Code wird mehrmals täglich integriert und getestet.
- **Refactoring**
Der Code wird kontinuierlich verbessert, ohne Funktionalität zu verändern.
- **Simple Design**
Der Code soll so einfach wie möglich sein – gerade genug, um zu funktionieren.
- **Collective Code Ownership**
Jeder darf jeden Teil des Codes ändern.
- **Coding Standards**
Das Team einigt sich auf gemeinsame Richtlinien für sauberen Code.



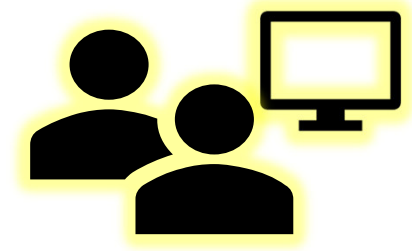
Quelle: https://de.wikipedia.org/wiki/Extreme_Programming

SOFTWAREENTWICKLUNG

PAIR PROGRAMMING

„**Pair programming** is a style of programming in which two programmers work side by side at one computer, continually collaborating on the same design, algorithm, code, or test.“

(Kent Beck, 1999, in Extreme Programming Explained: Embrace Change)



Vorteile:

- **Höhere Codequalität**

Zwei Augenpaare finden eher Fehler und verbessern den Code direkt beim Schreiben.

- **Wissensaustausch & Weiterbildung**

Weniger erfahrene Entwickler lernen von erfahreneren Kollegen ("learning by doing").

- **Bessere Problemlösung**

Zwei Personen bringen unterschiedliche Perspektiven und Ideen ein.

- **Konzentration & Disziplin**

Die Zusammenarbeit fördert Fokus und verhindert Ablenkungen.

- **Geringere Fehlerquote**

Bugs werden oft frühzeitig entdeckt – bevor sie teuer werden.

Nachteile:

- **Höherer Zeit- und Kostenaufwand**

Zwei Entwickler arbeiten an einer Aufgabe – das kann teurer erscheinen.

- **Ermüdung & soziale Belastung**

Ständige Interaktion kann ermüden oder zu Konflikten führen, besonders bei unpassenden Team-Dynamiken.

- **Nicht für alle Aufgaben geeignet**

Routineaufgaben oder einfache Tasks profitieren oft nicht vom Pairing.

- **Abhängigkeit vom Partner**

Wenn einer viel stärker oder schwächer ist, kann das Ungleichgewicht die Effizienz bremsen.

SOFTWAREENTWICKLUNG

MOB PROGRAMMING

"**Mob Programming** is all the brilliant minds working together on the same thing, at the same time, in the same space, and at the same computer."

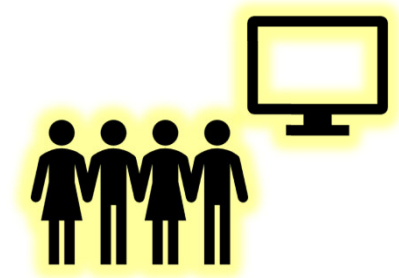
(Woody Zuill)

Vorteile:

- **Gemeinsames Wissen:**
Alle lernen voneinander – Wissen verteilt sich gut.
- **Bessere Codequalität:**
Mehr Augen, weniger Fehler.
- **Schnelle Abstimmung:**
Entscheidungen werden direkt im Team getroffen.
- **Stärkeres Teamgefühl:**
Fördert Zusammenarbeit und Vertrauen.

Nachteile:

- **Hoher Aufwand:**
Alle arbeiten an einer Aufgabe – das wirkt ineffizient.
- **Ermüdung:**
Längere Sessions können anstrengend sein.
- **Nicht für alles geeignet:**
Simple Aufgaben lohnen sich oft nicht im Team.



Softwareentwicklung Extreme Programming (XP)

Übungsfragen



1. Was wird als **Extreme Programming (XP)** bezeichnet?
2. Welche **Kernpraktiken** werden beim **Extreme Programming (XP)** angewendet?
3. Worin unterscheidet sich **Pair Programmig** von **Mob Programming**?
4. Was sind mögliche **Vorteile** des **Pair Progammmings**?
5. Was sind mögliche **Nachteile** des **Pair Progammmings**?

Softwareentwicklung

Lernziele



Nach der Bearbeitung dieser Lektion werdet ihr wissen, ...

- wie sich **klassische** und **agile Projektmanagement-Ansätze** in der **Softwareentwicklung** unterscheiden.
- wie sich **agile Projektmanagement-Ansätze** in der **Softwareentwicklung anwenden** lassen (bspw. **Kanban, Scrum, Scrumban**).
- was **Extreme Programming (XP)** bedeutet und welche **Kernpraktiken** dazugehören.
- wie sich **Extreme Programming (XP)** in Data Science Projekten **anwenden** lässt.

Softwareentwicklung Extreme Programming (XP)

30 Minuten Zeit

Aufgabe:

1. Bildet **Teams aus 2 Personen**.
2. Findet gemeinsam einen Namen für euer Team (hier ein Tool zur Unterstützung: <https://namemyteam.com/team-name-generator/>)
3. Erstellt auf Basis des **Datensatzes „cali_data.csv“** und eurer bereits durchgeführten **explorativen Datenanalyse (EDA)** ein Notebook mit einem **einfachen Vorhersagemodell** für Immobilien in Kalifornien das die Frage beantwortet: „**Welcher Preis ist ein Haus wert?**“.
 - Arbeitet als Team im Sinne des “Extreme Programming” (XP) nach der Kernpraktik “Pair Programming” (TDD und weitere Praktiken sind erstmal nicht notwendig!)
 - Ihr könnt euch an dem Beispiel aus dem Notebook „Cali_Housing.ipynb“ orientieren
 - Eine Evaluation des Modells ist nicht notwendig!

